Computer Methods in Applied Mechanics and Engineering 418 (2024) 116531

Contents lists available at ScienceDirect

Comput. Methods Appl. Mech. Engrg.

journal homepage: www.elsevier.com/locate/cma

Capturing the diffusive behavior of the multiscale linear transport equations by Asymptotic-Preserving Convolutional DeepONets

Keke Wu [a], Xiong-Bin Yan [a,c], Shi Jin [a,b,d], Zheng Ma [a,b,c,*]

[a] School of Mathematical Sciences, Shanghai Jiao Tong University, Shanghai, 200240, PR China
[b] Institute of Natural Sciences, MOE-LSC, Shanghai Jiao Tong University, Shanghai, 200240, PR China
[c] Qing Yuan Research Institute, Shanghai Jiao Tong University, Shanghai, 200240, PR China
[d] Shanghai Artificial Intelligence Lab, Shanghai, PR China

ARTICLE INFO

Keywords:
Asymptotic-preserving
Convolutional
DeepONets
Multiscale
Heat kernel

ABSTRACT

In this paper, we introduce two types of novel Asymptotic-Preserving Convolutional Deep Operator Networks (APCONs) designed to solve the multiscale time-dependent linear transport equations. We observe that the vanilla physics-informed DeepONets with modified MLP may exhibit instability in maintaining the desired limiting macroscopic behavior. Therefore, this necessitates the utilization of an asymptotic-preserving loss function. Drawing inspiration from the heat kernel in the diffusion equation, we propose a new architecture called Convolutional Deep Operator Networks, which employs multiple local convolution operations instead of a global heat kernel, along with pooling and activation operations in each filter layer. Our APCON methods possess a parameter count that is independent of the grid size and are capable of capturing the diffusive behavior of the linear transport problem. Finally, we validate the effectiveness of our methods through several numerical examples.

1. Introduction

Kinetic equations model the dynamics of particles that transport and collide between themselves or interact with the background media or external fields. These equations are defined in the phase space thus suffer from curse-of-dimensionality. In addition, they often possess multiple spatial and/or temporal scales, as well as non-local operators, posing notable hurdles for numerical simulations. For a comprehensive exploration of this topic, see [1–4].

Deep learning methods and deep neural networks (DNNs) have received considerable attention within the scientific community, primarily due to their remarkable ability in addressing the intricate complexities involved in solving partial differential equations (PDEs) [5–13]. For those interested in exploring alternative machine learning approaches to tackle partial differential equations, we mention some exemplary review articles [5,13]. These approaches are primarily motivated by the idea of parameterizing solutions or gradients of PDE problems using deep neural networks. As a result, they lead to a high-dimensional and non-convex minimization problem. Unlike traditional numerical methods, deep learning methods are mesh-free, enabling the solution of PDEs in intricate domains and geometries, and alleviating the issue of curse-of-dimensionality. The advantages lie in their flexibility and ease of implementation. However, it is important to note that deep learning methods come with certain potential drawbacks, such as lengthy training times, convergence issues, and reduced accuracy. While the majority of preceding neural network architectures excel in acquiring mappings between finite-dimensional Euclidean spaces, their capacity for generalization becomes constrained when confronted with disparate parameters, initial conditions, or boundary conditions. The concept of operator learning presents

* Corresponding author at: School of Mathematical Sciences, Shanghai Jiao Tong University, Shanghai, 200240, PR China.
E-mail address: zhengma@sjtu.edu.cn (Z. Ma).

https://doi.org/10.1016/j.cma.2023.116531
Received 15 August 2023; Received in revised form 26 September 2023; Accepted 9 October 2023
Available online 14 October 2023
0045-7825/© 2023 Elsevier B.V. All rights reserved.

a promising approach to solving a specific class of PDEs by training the neural network only *once*, see for example [14–23] and references therein. As an example, the Fourier neural operator (FNO) framework [14] exhibits the remarkable capability of proficiently acquiring the mapping between infinite-dimensional spaces through input–output pairs. However, it is crucial to acknowledge that there are still several unresolved aspects pertaining to the convergence theory.

In recent years, there has been a wealth of research dedicated to the utilization of deep neural networks in addressing multiscale kinetic equations and hyperbolic systems. This body of work encompasses a range of studies, some of which are referenced in the citations [23–32]. The importance of tackling kinetic problems that exhibit characteristics across multiple scales has been widely recognized. As we are aware, there exist numerous options to construct the loss when presented with a partial differential equation (PDE). For example, the variational formulation (DRM) [7], the least-squares formulation (PINN, DGM) [10,11], and the weak formulation (WAN) [12], among others. Due to the presence of small scales, the conventional Physics-Informed Neural Networks (PINNs) can perform poorly when dealing with multiscale kinetic equations [27,28]. Consequently, an alternative approach for solving multiscale kinetic equations utilizing Deep Neural Networks involves formulating a loss function that can effectively capture the macroscopic behavior at the limit—referred to as the Asymptotic-Preserving (AP) loss. A computational method for multiscale kinetic equations is called AP if it mimics the asymptotic transition from the kinetic equation to its macroscopic diffusive or hydrodynamic limit [3], which has been a standard criterion for developing multiscale kinetic solvers that can capture efficiently the macroscopic behavior even if the small physical scale – the Knudsen number – is not numerically resolved. This, in turn, justifies the necessity of employing Asymptotic-Preserving Neural Networks (APNNs) [27]. In our previous works [27,33], we presented two distinct methodologies employing APNNs for addressing time-dependent linear transport equations with diffusive scaling and uncertainties. These approaches are based on micro-macro and even–odd decomposition techniques, respectively. Furthermore, we provided empirical evidence demonstrating the AP characteristics of the loss function as the Knudsen number tends towards zero.

This paper focuses on *operator learning* for the multiscale time-dependent linear transport equation. The *operator learning* tasks include methods that endeavor to map infinite-dimensional function spaces by acquiring knowledge of solution operators through neural networks. These operators facilitate the mapping of input functions (e.g., initial conditions, boundary conditions, or forcing terms) to solutions of partial differential equations [14,15]. It has been observed that the vanilla physics-informed DeepONets, when equipped with a modified multi-layer perceptron, may encounter instability issues in preserving the desired macroscopic behavior. To address this, we first introduce the concept of an AP loss function in the context of operator learning. Subsequently, taking inspiration from the heat kernel in the diffusion equation, we propose a novel architecture named Convolutional DeepONets. These networks employ multiple local convolution operations, instead of a global heat kernel, along with pooling and activation operations in each filter layer. As a result, we introduce two innovative types of Asymptotic-Preserving Convolutional DeepONets (APCONs) based on micro-macro and even–odd decomposition – which are two popular AP schemes for multiscale linear transport equations with diffusive scaling – respectively. Notably, our APCON methods possess a parameter count that remains unaffected by the grid size, while effectively capturing the diffusive behavior inherent in the linear transport problem.

An outline of this paper is as follows. In Section 2, a detail illustration of the linear transport equations under the diffusion regime are given. Section 3 is our main part, we reviewed the physics-informed DeepONets and proposed two novel Asymptotic-Preserving Convolutional DeepONets for the multiscale time-dependent linear transport problem. Numerous numerical examples are presented in Section 4 to demonstrate the effectiveness of the APCONs. The paper is concluded in Section 5.

2. The linear transport equations under the diffusion regime

We investigate the linear transport problem within the framework of diffusive scaling, specifically in the context of one-dimensional space and velocity variables. The problem is formulated as follows:

$$\varepsilon\partial_t f + v\cdot\nabla_x f = \frac{1}{\varepsilon}\mathcal{L}f + \varepsilon Q, \tag{1}$$

with corresponding initial and boundary conditions.

In this context, the function $f(t,x,v)$ represents the density distribution of particles at a given time $t\in\mathcal{T} := [0,T]$, at a spatial location $x\in\mathcal{D} := [x_L, x_R]$, and moving in the direction $v\in\Omega := [-1,1]$. The parameter $\varepsilon > 0$ corresponds to the Knudsen number, the ratio of the mean free path to a characteristic length. Furthermore, $Q(t,x)$ denotes the source function, $\mathcal{L}$ is the linear collision operator given by

$$\mathcal{L}f = \frac{1}{2}\int_{-1}^{1} f\,\mathrm{d}v - f. \tag{2}$$

2.1. The micro-macro decomposition

In this section, we present the micro-macro decomposition formulation for the linear transport equation [34]. The approach involves decomposing the function f into two components: the equilibrium part denoted as $\rho(t,x)$ and the non-equilibrium part represented by $g(t,x,v)$:

$$f = \rho + \varepsilon g, \tag{3}$$

where

$$\rho := \langle f\rangle = \frac{1}{2}\int_{-1}^{1} f\,\mathrm{d}v. \tag{4}$$

The non-equilibrium component g satisfies the following relation:

$$\langle g \rangle = 0. \tag{5}$$

By applying Eq. (3) to Eq. (1), one obtains the following expression:

$$\varepsilon \partial_t \rho + \varepsilon^2 \partial_t g + v \cdot \nabla_x \rho + \varepsilon v \cdot \nabla_x g = \mathcal{L} g + \varepsilon Q. \tag{6}$$

By integrating this equation with respect to v, the following continuity equation with a source term can be derived:

$$\partial_t \rho + \nabla_x \cdot \langle v g \rangle = Q. \tag{7}$$

By defining the operator Π as $\Pi(\cdot)(v) = \langle \cdot \rangle$ and I as the identity operator, one obtains an evolution equation for g by applying the orthogonal projection $I - \Pi$ to Eq. (6).

$$\varepsilon^2 \partial_t g + \varepsilon (I - \Pi)(v \cdot \nabla_x g) + v \cdot \nabla_x \rho = \mathcal{L} g + (I - \Pi) \varepsilon Q. \tag{8}$$

The micro-macro formulation of Eq. (1) for our APCON consists of Eqs. (7) and (8), along with the constraints (4) and (5):

$$\begin{cases} \partial_t \rho + \nabla_x \cdot \langle v g \rangle = Q, \\ \varepsilon^2 \partial_t g + \varepsilon (I - \Pi)(v \cdot \nabla_x g) + v \cdot \nabla_x \rho = \mathcal{L} g + (I - \Pi) \varepsilon Q, \\ \langle g \rangle = 0. \end{cases} \tag{9}$$

As ε tends to 0, the aforementioned system formally approaches the following:

$$\begin{cases} \partial_t \rho + \nabla_x \cdot \langle v g \rangle = Q, \\ v \cdot \nabla_x \rho = \mathcal{L} g. \end{cases} \tag{10}$$

The second equation yields

$$g = \mathcal{L}^{-1}(v \cdot \nabla_x \rho), \tag{11}$$

upon substituting the expressions into the first equation and integrating over v, one obtains the diffusion Eq. (12):

$$\partial_t \rho = \frac{1}{3} \partial_{xx} \rho + Q\,. \tag{12}$$

2.2. The even–odd decomposition

An alternative approach, known as the even–odd decomposition, was introduced by Lewis and Miller [35]. This method involves splitting Eq. (1) into two separate equations, one for v and another for $-v$:

$$\varepsilon \partial_t f(v) + v \cdot \nabla_x f(v) = \frac{1}{\varepsilon} \mathcal{L} f(v) + \varepsilon Q, \tag{13a}$$

$$\varepsilon \partial_t f(-v) - v \cdot \nabla_x f(-v) = \frac{1}{\varepsilon} \mathcal{L} f(-v) + \varepsilon Q, \tag{13b}$$

and let us define even- and odd-parities as follows:

$$\begin{aligned} r(t,x,v) &= \frac{1}{2}[f(t,x,v) + f(t,x,-v)], \\ j(t,x,v) &= \frac{1}{2\varepsilon}[f(t,x,v) - f(t,x,-v)]. \end{aligned} \tag{14}$$

By adding and subtracting the two equations in Eq. (13), one can derive the following system of equations:

$$\begin{aligned} \partial_t r + v \cdot \nabla_x j &= \frac{1}{\varepsilon^2} \mathcal{L} r + Q, \\ \partial_t j + \frac{1}{\varepsilon^2} v \cdot \nabla_x r &= -\frac{1}{\varepsilon^2} j. \end{aligned} \tag{15}$$

The first equation by integrating over v gives

$$\partial_t \langle r \rangle + \langle v \cdot \nabla_x j \rangle = \frac{1}{\varepsilon^2}(\rho - \langle r \rangle) + Q, \tag{16}$$

and since $\rho = \langle r \rangle$, one can express it as follows:

$$\partial_t \rho + \langle v \cdot \nabla_x j \rangle = Q. \tag{17}$$

The even–odd formulation of Eq. (1) for our APCON consists of Eqs. (15) and (17), along with the constraint $\rho = \langle r \rangle$:

$$\begin{cases} \varepsilon^2 \partial_t r + \varepsilon^2 v \cdot \nabla_x j = \rho - r + \varepsilon^2 Q, \\ \varepsilon^2 \partial_t j + v \cdot \nabla_x r = -j, \\ \partial_t \rho + \langle v \cdot \nabla_x j \rangle = Q, \\ \rho = \langle r \rangle\,. \end{cases} \tag{18}$$

When $\varepsilon \to 0$, the above equation formally approaches

$$\begin{cases} r = \rho, \\ j = -v \cdot \nabla_x r, \\ \partial_t \rho + \langle v \cdot \nabla_x j \rangle = Q. \end{cases} \tag{19}$$

By substituting the first equation into the second equation, we obtain $j = -v\partial_x \rho$. Plugging this expression into the third equation leads to the diffusion Eq. (12).

2.3. The heat kernel of diffusion equation

A fundamental solution, commonly referred to as a heat kernel, is a solution of the diffusion or heat equation that corresponds to an initial condition where heat is concentrated at a specific point. These fundamental solutions are valuable in determining general solutions of the heat equation across specific domains. For further exploration of this topic, introductory treatments can be found in references such as [36,37].

Taking the one-dimensional case as an example, the Green's function serves as a solution to the initial value problem. By utilizing Duhamel's principle, this is equivalent to defining the Green's function as the solution to the first equation, where a delta function is employed.

$$\begin{cases} u_t - k u_{xx} = 0, \ (t, x) \in \mathbb{R}^+ \times \mathbb{R} \\ u(0, x) = \delta(x), \end{cases} \tag{20}$$

where $\delta(x)$ represents the Dirac delta function. The solution obtained through the Fourier transform corresponds to the fundamental solution, also known as the heat kernel of Eq. (20):

$$\Phi(t, x) = \frac{1}{\sqrt{4\pi k t}} \exp\left(-\frac{x^2}{4kt}\right). \tag{21}$$

The general solution of the one-variable heat equation with an initial condition $u(0, x) = g(x)$, where $(t, x) \in \mathbb{R}^+ \times \mathbb{R}$, can be obtained by employing a convolution operation, known as the Poisson formula:

$$u(t, x) = \int_{-\infty}^{\infty} \Phi(t, x - y) g(y) \mathrm{d}y. \tag{22}$$

In the case of multiple spatial variables, the fundamental solution solves the analogous problem, and the fundamental solution in n variables is obtained by taking the product of the fundamental solutions in each individual variable:

$$\Phi(t, \boldsymbol{x}) = \frac{1}{(4\pi k t)^{\frac{n}{2}}} \exp\left(-\frac{|\boldsymbol{x}|^2}{4kt}\right). \tag{23}$$

The general solution of the heat equation on $\mathbb{R}^d$ can be obtained through convolution, allowing one to solve the initial value problem with the condition $u(0, \boldsymbol{x}) = g(\boldsymbol{x})$, where $\boldsymbol{x}$ represents a vector in $\mathbb{R}^d$.

Consider the inhomogeneous case on the whole line as

$$\begin{cases} u_t - k u_{xx} = f(t, x), \ (t, x) \in \mathbb{R}^+ \times \mathbb{R} \\ u(0, x) = g(x), \end{cases} \tag{24}$$

with $f(t, x), g(x)$ arbitrary given functions. For example, if $u(t, x)$ represents the temperature distribution along a rod, then $g(x)$ corresponds to the initial temperature distribution along the rod, and $f(t, x)$ represents a heat source (or sink) applied to the rod at later times.

Using Duhamel's principle, we can obtain the solution of Eq. (24) as follows:

$$u(t, x) = \int_{-\infty}^{\infty} \Phi(t, x - y) g(y) \mathrm{d}y + \int_0^t \int_{-\infty}^{\infty} \Phi(t - s, x - y) f(s, y) \mathrm{d}y \mathrm{d}s. \tag{25}$$

3. Operator learning for the linear transport equation

Let us consider the multiscale time-dependent linear transport equation with diffusive scaling, which can be expressed as follows:

$$\begin{cases} \varepsilon \partial_t f + v \cdot \nabla_x f = \frac{1}{\varepsilon} \mathcal{L} f, \ (t, x, v) \in \mathcal{T} \times \mathcal{D} \times \Omega, \\ \mathcal{B} f(t, x, v) = 0, \\ f(0, x, v) = f_0(x, v). \end{cases} \tag{26}$$

Our objective is to train a solution neural operator that maps the initial function $f_0(x, v)$ to the corresponding solutions of the PDE, denoted as $f(t, x, v)$.

It is important to note that the function space of $f_0(x, v)$ is infinite-dimensional. To represent it using neural networks, we need to utilize finite function values at specific points in order to construct a finite-dimensional approximation.

3.1. Neural networks

First, we introduce the notations for conventional multi-layer perceptron (MLP) networks.[1] An L-layer MLP is defined recursively as,

$$\begin{aligned} f^{[0]}(x) &= x, \\ f^{[l]}(x) &= \sigma\circ(W^{[l-1]} f^{[l-1]}(x) + b^{[l-1]}),\ 1 \le l \le L-1, \\ f_\theta(x) &= W^{[L-1]} f_\theta^{[L-1]}(x) + b^{[L-1]}, \end{aligned} \tag{27}$$

here, we denote $W^{[l]} \in \mathbb{R}^{m_{l+1}\times m_l}$ and $b^{[l]} \in \mathbb{R}^{m_{l+1}}$ as the weight matrix and bias vector, respectively, for the lth layer of the MLP. The dimensions m_0 and m_L correspond to the input and output dimensions of the network. The symbol "σ" represents a scalar nonlinear activation function. The notation "$\circ$" denotes an element-wise operation. We will use these notations consistently throughout the subsequent discussions for convenience, as long as it does not lead to any ambiguity.

In this paper, we employ the modified MLP architecture with layer normalization [38,39]. The architecture can be expressed as follows:

$$U = \sigma\circ(W_1^{[1]}x + b_1^{[1]}), V = \sigma\circ(W_2^{[1]}x + b_2^{[1]}), \tag{28}$$

$$H^{[1]} = \sigma\circ(W^{[1]}x + b^{[1]}), \tag{29}$$

$$Z^{[l]} = \sigma\circ(\mathrm{LayerNorm}(W^{[l]}H^{[l]} + b^{[l]})), \quad l = 1,\dots,L-1, \tag{30}$$

$$H^{[l+1]} = (1 - Z^{[l]}) \odot U + Z^{[l]} \odot V, \quad l = 1,\dots,L-1, \tag{31}$$

$$f_\theta(x) = W^{[L]}H^{[L]} + b^{[L]}, \tag{32}$$

where $W_1^{[1]}, W_2^{[1]} \in \mathbb{R}^{m_1\times m_0}, b_1^{[1]}, b_2^{[1]} \in \mathbb{R}^{m_1}$ and $\odot$ denotes element-wise multiplication.

In the study by Wang et al. [38], the incorporation of residual connections and consideration of multiplicative interactions among the inputs were shown to result in enhanced predictive performance. Layer normalization indeed constitutes a valuable technique employed to normalize the distributions of intermediate layers. Through this approach, it facilitates the acquisition of smoother gradients, thus expediting the training process and culminating in heightened generalization accuracy.

3.2. Physics-informed DeepONets

The concept of learning operators through a parametric-based approach was introduced in [40], where the authors proposed a method utilizing a one-layer MLP to learn non-linear operators. Furthermore, they presented a universal approximation theorem, guaranteeing that their architecture can approximate any continuous operator with arbitrary precision. Building upon this work, DeepONet was introduced in [15], employing a multi-layer MLP and demonstrating its efficacy in approximating solution operators for a wide range of ordinary and partial differential equations.

A DeepONet consists of two sub-networks: the branch net, responsible for encoding the input function $a(x)$ at a fixed number of sensors $x_i \in \mathbb{R}^d$, where $i = 1,\dots,m$; and the trunk net, responsible for encoding the locations y of the output functions. The objective of DeepONet is to learn an operator $G : a(y) \mapsto G(a)(y)$, which takes two inputs: $[a(x_1), a(x_2), \dots, a(x_m)]$ and $y \in \mathbb{R}^d$. Assuming that the branch net and trunk net output two vectors of length p: $[b_1, b_2, \dots, b_p]^T \in \mathbb{R}^p$ and $[t_1, t_2, \dots, t_p]^T \in \mathbb{R}^p$, respectively, the approximate output $G(a)(y)$ can be obtained as $\sum_{j=1}^p b_j t_j$. Additionally, a bias term $b_0 \in \mathbb{R}$ can be incorporated in the final stage if desired:

$$G(a)(y) \approx \sum_{j=1}^{p} b_j t_j + b_0. \tag{33}$$

In [18], an extension of DeepONet called physics-informed DeepONets (PIDONs) was introduced. PIDONs incorporate a regularization term into the loss function to enforce known physical constraints. This regularization enables the prediction of solutions to parametric differential equations, even in cases where paired input–output training data is not available. We have included a comprehensive schematic diagram illustrating the PIDON methodology employed for resolving the PDE in Fig. 1.

Physics-informed DeepONets leverage available data and/or physical constraints to acquire knowledge regarding the solution operator for a particular set of PDEs. By doing so, they are capable of surmounting the limitations associated with purely data-driven approaches and those solely based on physics principles. The neural network architecture employed in physics-informed DeepONets, as previously mentioned, involves a modified MLP [38] which has been specifically tailored for this purpose.

In the context of solving the linear transport equation using neural operators, the physics-informed DeepONet methodology employs the flattened and discretized initial function denoted as a, along with the location coordinates (t, x, v), as inputs for both the branch and trunk networks. These networks then generate the approximate neural solution denoted as f_θ, which is evaluated at the corresponding coordinates (t, x, v).

[1] BAAI.2020. Suggested Notation for Machine Learning. https://github.com/mazhengcn/suggested-notation-for-machine-learning.

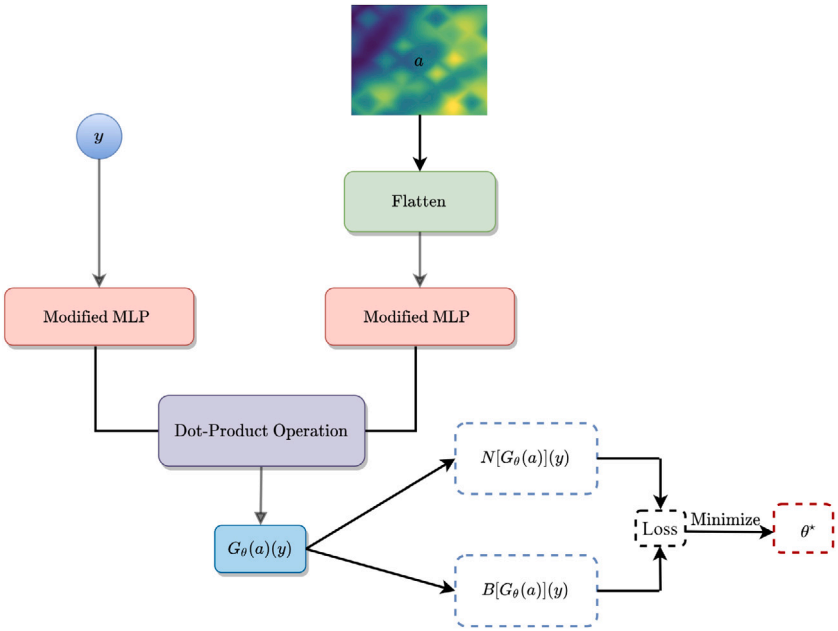


Fig. 1. The architecture of the Physics-informed DeepONets is characterized by its core component, the DeepONet with a modified MLP. This architecture includes two distinct sub-networks dedicated to extracting underlying representations of input functions denoted as a, and the corresponding coordinates denoted as y. Moreover, in relation to the equation component, the loss function for the PDE is denoted as $N[G_\theta(a)](y)$, while the loss function for the boundary, initial, and other observed data is denoted as $B[G_\theta(a)](y)$.

Consequently, a single DeepONet, equipped with a modified MLP, is utilized to construct the solution operator for a specific input denoted as a :

$$f_\theta[a](t,x,v) := \sigma_+(G_\theta(a)(t,x,v)) \approx f(t,x,v), \tag{34}$$

here $\sigma_+(\cdot) = \log(1+\exp(\cdot))$ is applied to keep $f_\theta[a](t,x,v)$ be positive.

The loss function of the linear transport equation for the Physics-informed DeepONet, considering a specific input denoted as a, can be expressed as follows:

$$\begin{aligned}\mathcal{L}^a_{\text{PIDON}} &= \frac{1}{|\mathcal{T}\times D\times\Omega|}\int_{\mathcal{T}\times D\times\Omega} |\varepsilon^2\partial_t f_\theta + \varepsilon v\cdot\nabla_x f_\theta - \mathcal{L} f_\theta|^2 \mathrm{d}t\mathrm{d}x\mathrm{d}v \\ &+ \frac{1}{|\mathcal{T}\times\Omega|}\int_{\mathcal{T}\times\Omega} |\mathcal{B} f_\theta[a](t,x,v) - 0|^2 \mathrm{d}t\mathrm{d}v \\ &+ \frac{1}{|D\times\Omega|}\int_{D\times\Omega} |f_\theta[a](0,x,v) - f_0(x,v)|^2 \mathrm{d}x\mathrm{d}v,\end{aligned} \tag{35}$$

where $|X|$ denotes the measure of the domain X. In the operator learning task, it is necessary to compute the average of $\mathcal{L}^a_{\text{PIDON}}$ across all samples $a \in \{a^i\}_{i=1}^M$. The corresponding total loss function is defined as follows:

$$\mathcal{L}_{\text{PIDON}} = \frac{1}{M}\sum_{i=1}^{M} \mathcal{L}^{a^i}_{\text{PIDON}}. \tag{36}$$

We put a schematic plot of PIDON method for the linear transport equation in Fig. 2.

3.3. Asymptotic-preserving convolutional DeepONets

3.3.1. Convolutional DeepONets

Although DeepONets still rely on large annotated datasets comprising paired input–output observations, they offer a straightforward and intuitive model that enables a continuous representation of the target output functions, regardless of resolution. However, it is worth noting that the sensors of the branch network in DeepONets typically operate on grid points within the domain in $\mathbb{R}^d$. As the dimensionality increases, this approach leads to an exponential growth in the number of grid points and consequently leads to a substantial increase in the size of neural parameters within the first layer of the branch network.

Taking inspiration from the macroscopic limit of the linear transport equation as $\varepsilon \to 0$ and the Poisson formula of the diffusion equation (Eqs. (22) and (25)), we propose a novel architecture called Convolutional DeepONets (CONs). This architecture utilizes

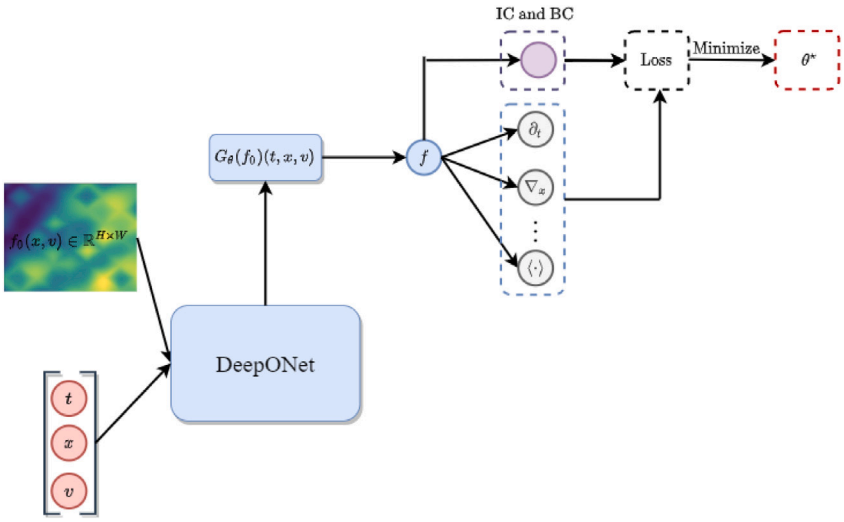


Fig. 2. The architecture of Physics-informed DeepONets for linear transport equation.

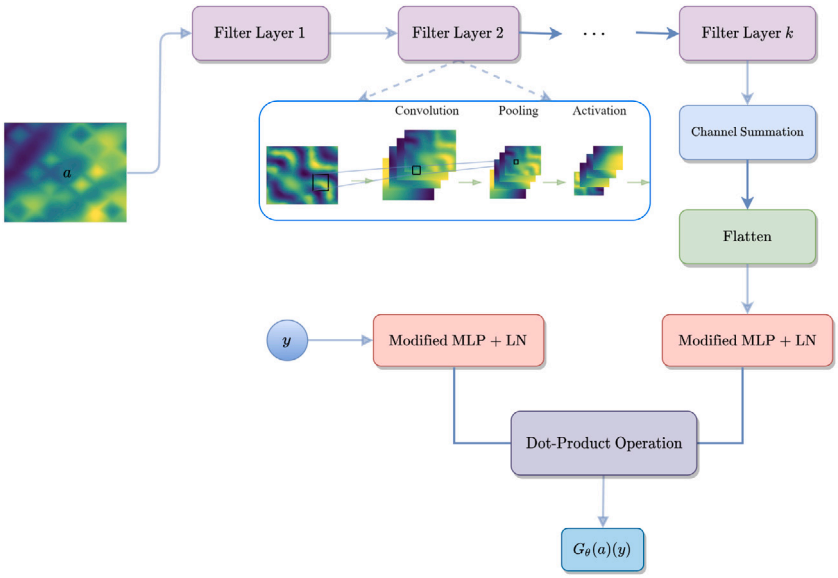


Fig. 3. The architecture of the 2D Convolutional DeepONets: (a) Branch net — Commence with an input function a having a discretized shape of $(H^{\text{in}}, W^{\text{in}})$. 1. Implement a series of filter layers, including convolution with C channels, pooling, and activation operations, with the resulting shape being $(C,\ H^{\text{out}}, W^{\text{out}})$. The order of pooling and activation operations can be adjusted as desired. 2. Sum the outputs from the filter layers across the channels, resulting in a flattened output of shape $(1,\ H^{\text{out}}W^{\text{out}})$. 3. Elevate the flattened output to a higher-dimensional space using a one-layer MLP and feed it into the modified MLP with layer normalization. (b) Trunk net — Begin with a single point denoted as y. Apply the modified MLP with layer normalization to process the input.

a sequence of convolution, pooling, and activation operations (referred to as filter layers) for the inputs. Following these filter layers, we perform a summation along the channels and flatten the resulting output. Optionally, one can perform pooling and activation operations in a different order. Subsequently, the output is passed through the modified MLP with layer normalization. The underlying concept behind this approach is to employ multiple local convolution operations instead of a global heat kernel, thereby reducing the parameter size of the branch network. In addition, we apply the modified MLP with layer normalization for the trunk network. Notably, the number of parameters in our Convolutional DeepONets is not dependent on the grid size. The architecture of the 2D Convolutional DeepONets is illustrated in Fig. 3. It is important to mention that this diagram depicts the specific structure for the 2D case, while a more general structure for higher-dimensional cases can be constructed following a similar analogy.

In the study conducted in [20], the authors introduced a model called MultiAuto-DeepONet. This model utilizes an encoder–decoder architecture to handle high-dimensional input random processes. The dimensionality reduction process in their model is

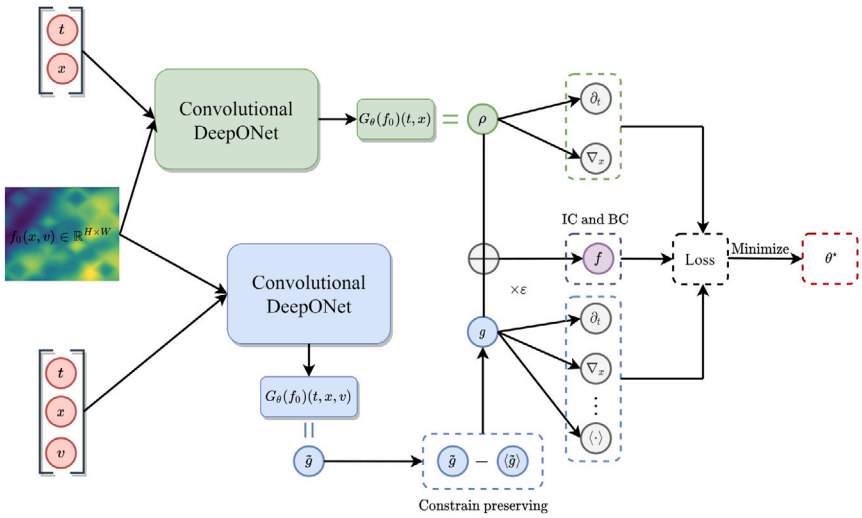


Fig. 4. The architecture of Asymptotic-Preserving Convolutional DeepONets based on the micro-macro decomposition for linear transport equation.

achieved solely through convolutional layers acting as encoders, in contrast to our approach which incorporates both convolutional layers and a vanilla DeepONet with MLP. The difference arises from the intention to capture the diffusive characteristics of multiscale time-dependent linear transport equations in our model.

3.3.2. APCONs based on the micro-macro decomposition

The APCON method based on micro-macro decomposition utilizes the discretized initial function a along with the location coordinates (t, x, v) or (t, x) as inputs for both the branch and trunk networks. The method then generates the approximate neural solutions ρ_θ evaluated at (t, x), while g_θ is evaluated at (t, x, v).

To construct the solution operator for ρ and g with respect to a specific input a, two Convolutional DeepONets are employed. These Convolutional DeepONets are utilized to model and approximate the solution operators for ρ and g respectively,

$$\begin{aligned} \rho_\theta[a](t,x) &:= G_\theta(a)(t,x) \approx \rho(t,x), \\ g_\theta[a](t,x,v) &:= G_\theta(a)(t,x,v) - \langle G_\theta(a)(t,x,v)\rangle \approx g(t,x,v). \end{aligned} \tag{37}$$

The loss function of the linear transport equation for APCON, based on micro-macro decomposition and considering a specific input a, can be expressed as follows:

$$\begin{aligned} \mathcal{L}^a_{\text{APCON-v1}} = &\frac{1}{|\mathcal{T}\times D|}\int_{\mathcal{T}\times D} |\partial_t \rho_\theta + \langle v\cdot\nabla_x g_\theta\rangle|^2 \mathrm{d}t\mathrm{d}x \\ &+ \frac{1}{|\mathcal{T}\times D\times\Omega|}\int_{\mathcal{T}\times D\times\Omega} |\varepsilon^2\partial_t g_\theta + \varepsilon(I-\Pi)(v\cdot\nabla_x g_\theta) + v\cdot\nabla_x\rho_\theta - \mathcal{L}g_\theta|^2 \mathrm{d}t\mathrm{d}x\mathrm{d}v \\ &+ \frac{1}{|\mathcal{T}\times\Omega|}\int_{\mathcal{T}\times\Omega} |\mathcal{B}(\rho_\theta+\varepsilon g_\theta)[a](t,x,v) - 0|^2 \mathrm{d}t\mathrm{d}v \\ &+ \frac{1}{|D\times\Omega|}\int_{D\times\Omega} |(\rho_\theta+\varepsilon g_\theta)[a](0,x,v) - f_0(x,v)|^2 \mathrm{d}x\mathrm{d}v. \end{aligned} \tag{38}$$

In the aforementioned construction, the constraint (5) is automatically omitted, similar to the approach described in the work [27].

In the operator learning task, we compute the average of $\mathcal{L}^a_{\text{APCON-v1}}$ across all samples $a \in \{a^i\}_{i=1}^M$ to obtain the corresponding total loss function, which is defined as follows:

$$\mathcal{L}_{\text{APCON-v1}} = \frac{1}{M}\sum_{i=1}^{M} \mathcal{L}^{a^i}_{\text{APCON-v1}}. \tag{39}$$

We put a schematic plot of our APCON method based on micro-macro decomposition for the linear transport equation in Fig. 4.

3.3.3. APCONs based on the even–odd decomposition

The APCON method, based on even–odd decomposition, utilizes the discretized initial function a along with the location coordinates (t, x, v) or (t, x) as inputs for both the branch and trunk networks. The method then generates the approximate neural solutions ρ_θ evaluated at (t, x), while r_θ and j_θ are evaluated at (t, x, v).

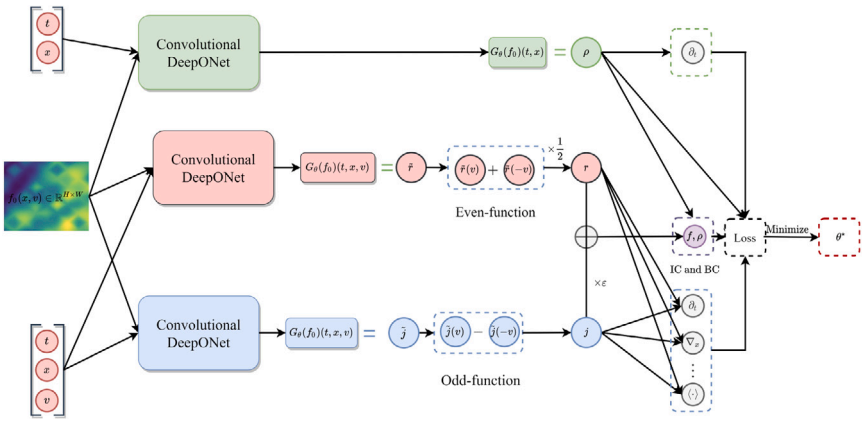


Fig. 5. The architecture of Asymptotic-Preserving Convolutional DeepONets based on even–odd decomposition for linear transport equation.

To construct the solution operator for ρ, r and j with respect to a specific input a, three Convolutional DeepONets are employed. These Convolutional DeepONets are utilized to model and approximate the solution operators for ρ, r and j respectively,

$$\begin{aligned}
\rho_\theta[a](t,x) &:= G_\theta(a)(t,x) \approx \rho(t,x),\\
r_\theta[a](t,x,v) &:= \frac{1}{2}\left(G_\theta(a)(t,x,v) + G_\theta(a)(t,x,-v)\right) \approx r(t,x,v),\\
j_\theta[a](t,x,v) &:= G_\theta(a)(t,x,v) - G_\theta(a)(t,x,-v) \approx j(t,x,v).
\end{aligned} \tag{40}$$

Here $r_\theta[a](t,x,v)$ and $j_\theta[a](t,x,v)$ satisfy the even–odd property.

The loss function of the linear transport equation for APCON, based on the even–odd parity formulation and considering a specific input a, can be expressed as follows:

$$\begin{aligned}
\mathcal{L}^a_{\text{APCON-v2}} = &\frac{1}{|\mathcal{T}\times D\times\Omega|}\int_{\mathcal{T}\times D\times\Omega} |\varepsilon^2\partial_t r_\theta + \varepsilon^2 v\cdot\nabla_x j_\theta - (\rho_\theta - r_\theta)|^2 \mathrm{d}t\mathrm{d}x\mathrm{d}v\\
&+ \frac{1}{|\mathcal{T}\times D\times\Omega|}\int_{\mathcal{T}\times D\times\Omega} |\varepsilon^2\partial_t j_\theta + v\cdot\nabla_x r_\theta - (-j_\theta)|^2 \mathrm{d}t\mathrm{d}x\mathrm{d}v\\
&+ \frac{1}{|\mathcal{T}\times D|}\int_{\mathcal{T}\times D} |\partial_t\rho_\theta + \langle v\cdot\nabla_x j_\theta\rangle|^2 \mathrm{d}t\mathrm{d}x\\
&+ \frac{1}{|\mathcal{T}\times D|}\int_{\mathcal{T}\times D} |\rho_\theta - \langle r_\theta\rangle|^2 \mathrm{d}t\mathrm{d}x\\
&+ \frac{1}{|\mathcal{T}\times\Omega|}\int_{\mathcal{T}\times\Omega} |\mathcal{B}(r_\theta + \varepsilon j_\theta)[a](t,x,v) - 0|^2 \mathrm{d}t\mathrm{d}v\\
&+ \frac{1}{|D\times\Omega|}\int_{D\times\Omega} |(r_\theta + \varepsilon j_\theta)[a](0,x,v) - f_0(x,v)|^2 \mathrm{d}x\mathrm{d}v.
\end{aligned} \tag{41}$$

In this operator learning task, we compute the average of $\mathcal{L}^a_{\text{APCON-v2}}$ across all samples $a \in \{a^i\}_{i=1}^M$ to obtain the corresponding total loss function, which is defined as follows:

$$\mathcal{L}_{\text{APCON-v2}} = \frac{1}{M}\sum_{i=1}^{M} \mathcal{L}^{a^i}_{\text{APCON-v2}}. \tag{42}$$

We put a schematic plot of our APCON method based on even–odd decomposition for the linear transport equation in Fig. 5.

4. Numerical experiments

The linear transport equations under consideration are accompanied by either inflow or Dirichlet boundary conditions.

Initially, we model random input functions $f_0(x,v)$ by employing a mean-zero Gaussian random field (GRF) with an exponential quadratic covariance kernel:

$$k(\boldsymbol{z},\boldsymbol{y}) = \exp\left(-\frac{\|\boldsymbol{z}-\boldsymbol{y}\|^2}{2l^2}\right),\ l > 0, \tag{43}$$

and, subsequently, we enforce the boundary conditions of the random input functions. In this process, we sample $M = 2^{10}$ distinct discretized initial functions with a shape of $(H,\ W) = (32,\ 64)$ using the GRF method with a covariance kernel parameter of $l = 1/2$. These sampled functions are then divided into training and testing datasets using a split rate of $7:1$. Furthermore, we approximate the loss using the Monte Carlo method. Specifically, the integration terms involving $\langle\cdot\rangle$ are computed using the Gauss–Legendre

quadrature rule with a total of 32 nodes. During each iteration of an epoch, the batch size for the input function a from the training dataset is set to $B = 4$, resulting in a shape of (B, H, W). Additionally, the number of sample points in the interior domain is $N_{\text{int}} = 2^{10}$, while the number of sample points on the boundary is $N_{\text{bdy}} = 2^8$. As for the initial loss, we select the coordinates of the discretized initial functions as sample points, with a total size of N_0. Therefore, for instance, when the inputs consist of a batch of discretized initial functions with a shape of (B, H, W), and the collocations of coordinates have a shape of $(N_{\text{int}}, 3)$, the outputs correspond to the approximate solutions at these coordinates for the B discretized initial functions, resulting in a shape of (B, N_{int}). The neural networks are trained for a total of 5000 epochs by minimizing the loss using the Adam optimizer [41]. The initial learning rate is set to 10^{-4}, and an annealing schedule is applied with a decay rate of 0.96 for every 100 epochs. For the modified MLP with layer normalization, the branch nets consist of $L = 5$ hidden layers, while the trunk nets consist of $L = 4$ hidden layers. Each hidden layer contains 64 units. The activation function used for all hidden layers in the modified MLP without layer normalization is swish(x). In most of the numerical experiments, we employ two filter layers, where each layer includes a convolution operation with 4 channels, a kernel shape of $(2, 2)$, and a stride shape of $(2, 2)$. Additionally, an average pooling operation with a shape of $(2, 2)$ and the same shape stride is applied, followed by an activation function with gelu(x). To succinctly convey the rationale, we refer to DeepONet as DON and Convolutional DeepONet as CON.

The empirical risk associated with PICON and APCON methods, which utilize micro-macro and even–odd decompositions for the linear transport equation, is

$$\begin{aligned}\mathcal{R}_{\text{PICON}} = \frac{1}{M}\sum_{i=1}^{M}\Big[& \frac{1}{N_{\text{int}}}\sum_{j=1}^{N_{\text{int}}} \left|\left(\varepsilon^2\partial_t f_\theta + \varepsilon v\cdot\nabla_x f_\theta - \mathcal{L} f_\theta\right)[a^i](t^j, x^j, v^j)\right|^2 \\ & + \frac{1}{N_{\text{bdy}}}\sum_{j=1}^{N_{\text{bdy}}} \left|\left(\mathcal{B} f_\theta\right)[a^i](t^j, x^j, v^j)\right|^2 \\ & + \frac{1}{N_0}\sum_{j=1}^{N_0} \left|\left(f_\theta - f_0\right)[a^i](0, x^j, v^j)\right|^2 \Big],\end{aligned} \tag{44}$$

$$\begin{aligned}\mathcal{R}_{\text{APCON-v1}} = \frac{1}{M}\sum_{i=1}^{M}\Big[& \frac{1}{N_{\text{int}}}\sum_{j=1}^{N_{\text{int}}} \left|\left(\partial_t \rho_\theta + \langle v\cdot\nabla_x g_\theta\rangle\right)[a^i](t^j, x^j)\right|^2 \\ & + \frac{1}{N_{\text{int}}}\sum_{j=1}^{N_{\text{int}}} \left|\left(\varepsilon^2\partial_t g_\theta + \varepsilon(I-\Pi)(v\cdot\nabla_x g_\theta) + v\cdot\nabla_x\rho_\theta - \mathcal{L} g_\theta\right)[a^i](t^j, x^j, v^j)\right|^2 \\ & + \frac{1}{N_{\text{bdy}}}\sum_{j=1}^{N_{\text{bdy}}} \left|\left(\mathcal{B}(\rho_\theta + \varepsilon g_\theta)\right)[a^i](t^j, x^j, v^j)\right|^2 \\ & + \frac{1}{N_0}\sum_{j=1}^{N_0} \left|\left((\rho_\theta + \varepsilon g_\theta) - f_0\right)[a^i](0, x^j, v^j)\right|^2 \Big],\end{aligned} \tag{45}$$

and

$$\begin{aligned}\mathcal{R}_{\text{APCON-v2}} = \frac{1}{M}\sum_{i=1}^{M}\Big[& \frac{1}{N_{\text{int}}}\sum_{j=1}^{N_{\text{int}}} \left|\left(\varepsilon^2\partial_t r_\theta + \varepsilon^2 v\cdot\nabla_x j_\theta - (\rho_\theta - r_\theta)\right)[a^i](t^j, x^j, v^j)\right|^2 \\ & + \frac{1}{N_{\text{int}}}\sum_{j=1}^{N_{\text{int}}} \left|\left(\varepsilon^2\partial_t j_\theta + v\cdot\nabla_x r_\theta - (-j_\theta)\right)[a^i](t^j, x^j, v^j)\right|^2 \\ & + \frac{1}{N_{\text{int}}}\sum_{j=1}^{N_{\text{int}}} \left|\left(\partial_t \rho_\theta + \langle v\cdot\nabla_x j_\theta\rangle\right)[a^i](t^j, x^j)\right|^2 \\ & + \frac{1}{N_{\text{int}}}\sum_{j=1}^{N_{\text{int}}} \left|\left(\rho_\theta - \langle j_\theta\rangle\right)[a^i](t^j, x^j)\right|^2 \\ & + \frac{1}{N_{\text{bdy}}}\sum_{j=1}^{N_{\text{bdy}}} \left|\left(\mathcal{B}(r_\theta + \varepsilon j_\theta)\right)[a^i](t^j, x^j, v^j)\right|^2 \\ & + \frac{1}{N_0}\sum_{j=1}^{N_0} \left|\left((r_\theta + \varepsilon j_\theta) - f_0\right)[a^i](0, x^j, v^j)\right|^2 \Big],\end{aligned} \tag{46}$$

respectively.

The reference solutions are obtained by fast spectral method and we will check the relative ℓ^2 error of the density solution $\rho(t, x)$ at all uniform grids $\{(t_i, x_j)\}_{1\le i\le I, 1\le j\le H}$,

$$\text{error} := \sqrt{\frac{\sum_{i,j}|\rho_\theta(t_i, x_j) - \rho_{\text{ref}}(t_i, x_j)|^2}{\sum_{i,j}|\rho_{\text{ref}}(t_i, x_j)|^2}}, \tag{47}$$

Table 1
Comparison in kinetic regime ($\varepsilon = 1$) of Problem I.

Method / QoIs	PI		AP v1		AP v2	
	DON	CON	DON	CON	DON	CON
Relative ℓ^2 error	1.88 e-2	1.74 e-2	3.32 e-2	1.59 e-2	1.48 e-2	1.44 e-2
# params	423 296	43 288	846 400	86 384	1 269 696	129 672

Table 2
Comparison in the diffusion regime ($\varepsilon = 10^{-4}$) of Problem I.

Method / QoI	PI		AP v1		AP v2	
	DON	CON	DON	CON	DON	CON
Relative ℓ^2 error	–	–	2.63e-2	1.58e-2	4.09e-2	1.55 e-2

where ρ_θ is the neural operator solution approximation, and ρ_{ref} is the reference solution.

Subsequently, we will conduct several experiments to compare the performance of PIDONs/CONs and APDONs/CONs utilizing the micro-macro and even–odd decompositions. The relative ℓ^2 error below is the optimal result obtained after 5 trials in all experiments.

4.1. Problem I: Inflow condition

Consider the linear transport equation in a boundary domain $\mathcal{T} \times \mathcal{D} \times \Omega = [0, T] \times [0, 1] \times [-1, 1]$:

$$\varepsilon \partial_t f + v \cdot \nabla_x f = \frac{1}{\varepsilon}\left(\frac{1}{2}\int_{-1}^{1} f \, \mathrm{d}v - f\right), \tag{48}$$

with in-flow boundary conditions as

$$\begin{aligned} f(t, x_L, v) &= 1, \quad \text{for} \quad v > 0, \\ f(t, x_R, v) &= \frac{1}{2}, \quad \text{for} \quad v < 0. \end{aligned} \tag{49}$$

Our focus is on the operator learning task that maps the initial function $f_0(x, v)$ to the solution $f(t, x, v)$ for the entire domain $\mathcal{T} \times \mathcal{D} \times \Omega$. Initially, to generate initial conditions that comply with the inflow boundary condition, we sample $\tilde{f}_0(x, v) \sim$ GRF and select positive samples to construct

$$f_0(x, v) = \left(\mathrm{relu}^3(v) \cdot x + \mathrm{relu}^3(-v) \cdot (1 - x)\right) \cdot \tilde{f}_0(x, v) + (1 - \frac{1}{2}x). \tag{50}$$

4.1.1. Kinetic regime with $\varepsilon = 1$

We plot the density predictions made by PIDON, APDON-v1, APDON-v2 and PICON, APCON-v1, APCON-v2 for a representative input function from the test datasets, alongside the reference solution in Figs. 6 and 7, respectively.

Table 1 records the relative ℓ^2 error of the density solution and the number of parameters for these methods. Our quantities of interest (QoIs) include the relative ℓ^2 error and the number of parameters for these neural networks. The relative l^2 error during the training of PIDON and APDON-v2 in the kinetic regime ($\varepsilon = 1$) is depicted in Fig. 8. It is evident that PIDON converges more swiftly than our method, primarily owing to its reduced parameter count and the lack of diffusion effects in this scenario.

4.1.2. Diffusion regime with $\varepsilon = 10^{-4}$

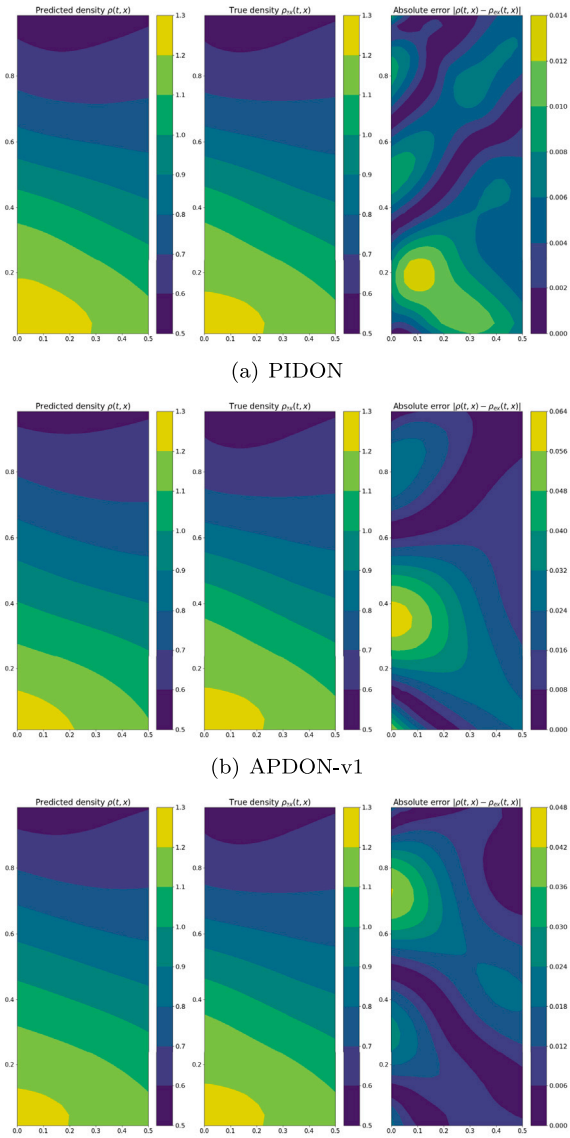
Table 2 records the relative ℓ^2 error of the density solution for these methods. The number of parameters for these methods is the same as previously mentioned. Here, a "-" indicates that the training process does not converge after reaching the maximum number of training epochs. This suggests that the loss function of operator learning in the form of vanilla PINN disregards the asymptotic property of the problem's multiscale nature and fails to capture the macroscopic behavior corresponding to small physical parameters, as explained in [27].

We plot the density predictions made by APCON-v1 and APCON-v2 for two representative input functions from the test datasets, alongside the reference solution in Fig. 9.

4.2. Problem II: Dirichlet condition

Consider the linear transport equation in a boundary domain $[0, 0.1] \times [0, 1] \times [-1, 1]$ with $\varepsilon = 10^{-4}$:

$$\varepsilon \partial_t f + v \cdot \nabla_x f = \frac{1}{\varepsilon}\left(\frac{1}{2}\int_{-1}^{1} f \, \mathrm{d}v - f\right), \tag{51}$$



(a) PIDON

(b) APDON-v1

(c) APDON-v2

Fig. 6. Plots of density $\rho(t,x)$ in $(t,x) \in [0,0.5]\times[0,1]$ by PIDON, APDON-v1, APDON-v2 for a representative input function. The results are obtained by DeepONets equipped with modified MLP.

with the boundary conditions $f(t,x_L,v) = 0 = f(t,x_R,v)$. The initial function is chosen from a distinct function space with a Maxwellian form as

$$f_0 = r \cdot \left[1 + \sin(2\pi x - \frac{\pi}{2})\right] \cdot 3\exp\left(-\frac{(3v)^2}{2}\right), \tag{52}$$

where r is sampled from $[0,1]$.

We plot the density predictions made by APCON-v1 and APCON-v2 for two representative input functions from the test datasets, alongside the reference solution in Fig. 10.

Table 3 records the relative ℓ^2 error of the density solution for these methods.

4.3. Computational cost

In this section, we present the comparative analysis of computational costs between APCONs and a numerical solver for the diffusion regime ($\varepsilon = 10^{-4}$) in Problem I. Our APCONs are implemented using JAX with the Haiku library, while the numerical

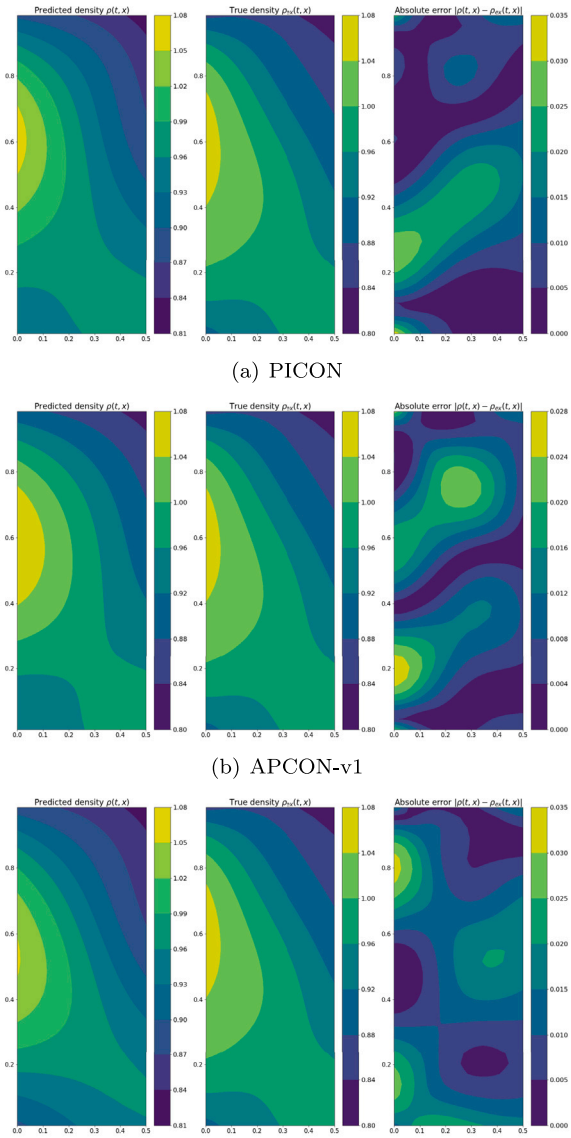


Fig. 7. Plots of density $\rho(t,x)$ in $(t,x) \in [0,0.5]\times[0,1]$ by PICON, APCON-v1, APCON-v2 method for a representative input function. The results are obtained by convolutional DeepONets equipped with modified MLP and layer normalization.

Table 3
Comparison in the diffusion regime ($\varepsilon = 10^{-4}$) of Problem II.

QoI \ Method	APCON-v1	APCON-v2
Relative ℓ^2 error	1.67 e-2	1.62 e-2

solver is a fast spectral method with a scalable implementation on a graphics processing unit (GPU) using cupy. As depicted in Fig. 11, our APCON methods demonstrate exceptional performance by accurately predicting the solutions and the average time is about 2 ms. It is worth noting that, even though the APCON methods offer an advantage in terms of inference speed, the training process for APCON methods tends to be relatively time-consuming. In this case, the training time for APCON-v1 and APCON-v2 amounts to 49907 and 46783 s, respectively. This represents a remarkable advancement, surpassing the capabilities of conventional PDE solvers by up to two orders of magnitude. Moreover, the inference process with APCONs is effortlessly parallelizable, enabling the prediction of solutions for an even larger number of equations using multiple GPUs.

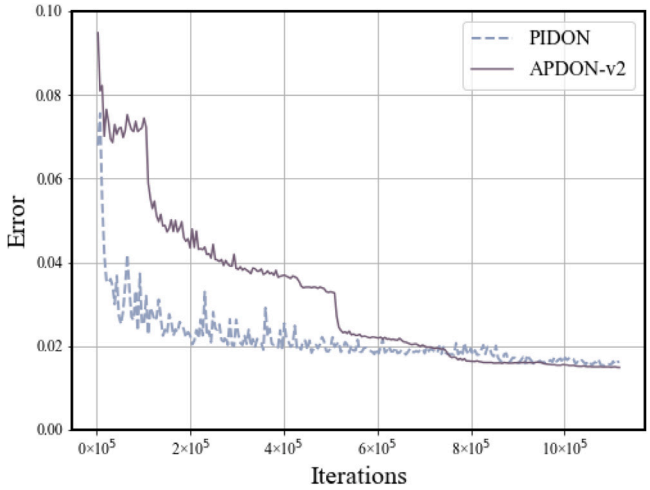
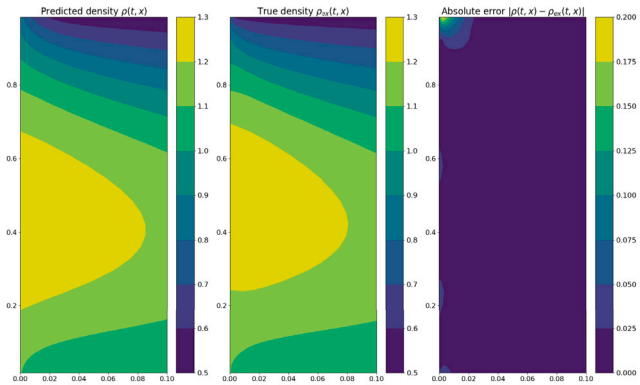
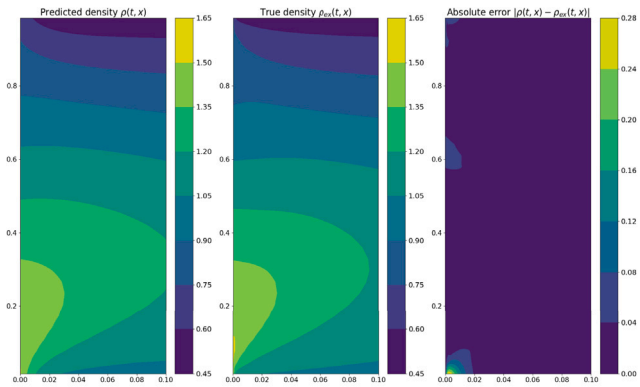


Fig. 8. The relative l^2 error of PIDON and APDON-v2 in the kinetic regime ($\epsilon = 1$) of Problem I.



(a) APCON-v1

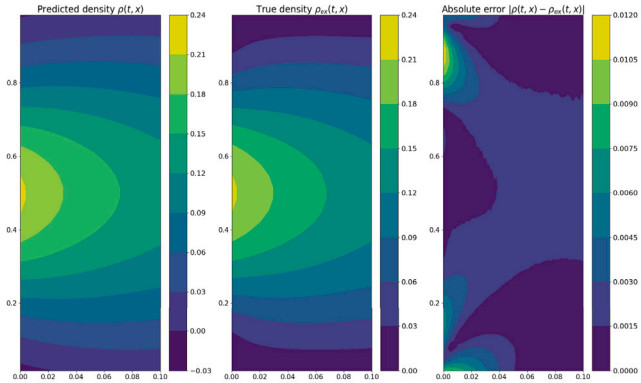


(b) APCON-v2

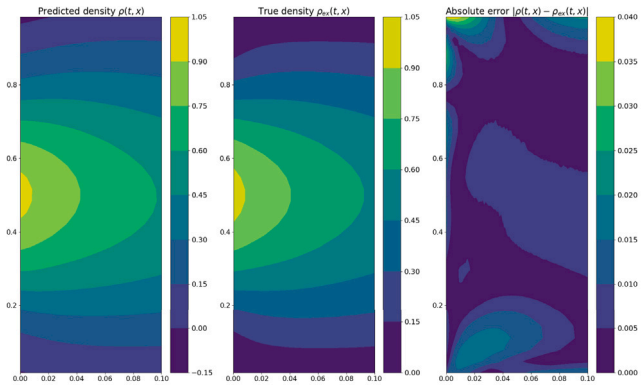
Fig. 9. Plots of density $\rho(t,x)$ in $(t,x) \in [0,0.1] \times [0,1]$ by APCON-v1 and APCON-v2 method for two representative input functions. The results are obtained by convolutional DeepONets equipped with modified MLP and layer normalization.

4.4. Layer normalization

In this section, we investigate the relative ℓ^2 error without layer normalization for the diffusion regime ($\epsilon = 10^{-4}$) of Problem I. Table 4 documents the relative ℓ^2 error of the density solution. It is worth noting that the utilization of layer normalization has the potential to enhance the precision of APCON-v2.



(a) APCON-v1



(b) APCON-v2

Fig. 10. Plots of density $\rho(t, x)$ in $(t, x) \in [0, 0.1] \times [0, 1]$ by APCON-v1 and APCON-v2 method for two representative input functions. The results are obtained by convolutional DeepONets equipped with modified MLP and layer normalization.

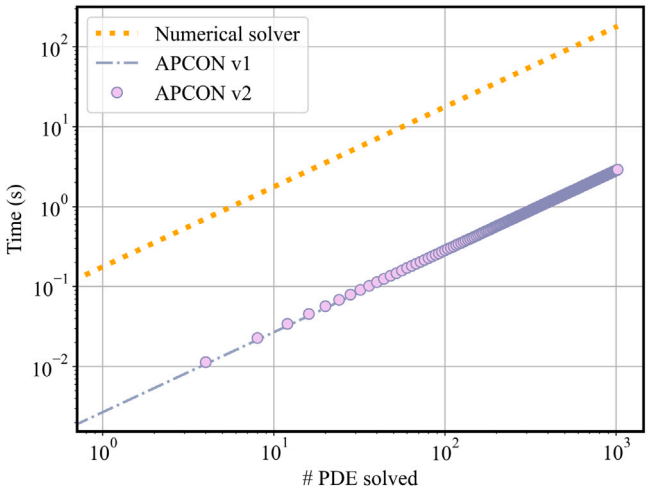


Fig. 11. Computational cost (second) for performing inference with pre-trained APCONs, along with the corresponding time required to solve a PDE with a conventional fast spectral method which encompasses a scalable GPU implementation utilizing cupy. Reported timings are obtained on a single NVIDIA RTX 3090 graphics processing unit (GPU). The average total time costs for the classical numerical solver, APCON-v1, and APCON-v2 are 176.9 ms, 2.69 ms, and 2.85 ms, respectively.

Table 4
APCON-v2 w/o LayerNorm for the diffusion regime ($\varepsilon = 10^{-4}$) of Problem I.

QoI \ Method	without LayerNorm	with LayerNorm
Relative ℓ^2 error	2.38 e-2	1.55 e-2

Table 5
The order of pooling and activation for the kinetic regime ($\varepsilon = 1$) of Problem I.

QoI \ Order	Activation + Pooling	Pooling + Activation
Relative ℓ^2 error	1.66 e-2	1.59 e-2

Table 6
The kernel size of convolutional layer for the kinetic regime ($\varepsilon = 10^{-4}$) of Problem I.

QoI \ Kernel size	$(1,2)$	$(2,2)$	$(2,4)$
Relative ℓ^2 error	2.52 e-2	1.55 e-2	2.51 e-2

Table 7
The number of channels for the kinetic regime ($\varepsilon = 10^{-4}$) of Problem I.

QoI \ # Channels	2	4	6
Relative ℓ^2 error	2.78 e-2	1.55 e-2	2.18 e-2

Table 8
The number of filter layers for the kinetic regime ($\varepsilon = 10^{-4}$) of Problem I.

QoI \ # Filter layers	1	2
Relative ℓ^2 error	2.30 e-2	1.55 e-2

4.5. Order of pooling and activation

Within the filter layers, it is possible to perform the activation step prior to the pooling operation. In this specific scenario, we are examining the sequencing of pooling and activation operations for the kinetic regime ($\varepsilon = 1$) of Problem I, employing APCON-v1.

Table 5 records the relative ℓ^2 error of the density solution pertaining to the ordering of pooling and activation, as observed through the utilization of APCON-v1. It is evident from the table that the impact on final accuracy in this particular case is negligible.

4.6. Kernel shape and channels

We shall now proceed to investigate the impact of kernel size and channels in the convolutional layer of APCON-v2, specifically concerning the diffusion regime ($\varepsilon = 10^{-4}$) of Problem I.

Tables 6 and 7 document the relative ℓ^2 error of the density solution for different kernel shapes, namely $(1,2)$, $(2,2)$ and $(2,4)$, as well as varying numbers of channels, namely 2, 4, and 6. All other settings remain unchanged. It is evident that the kernel size of $(2,2)$ with 4 channels demonstrates superior performance.

4.7. Filter layers

Lastly, we investigate the impact of the number of filter layers on the diffusion regime ($\varepsilon = 10^{-4}$) of Problem I. Table 8 illustrates the relative ℓ^2 error of the density solution when utilizing a single and two filter layers compared to other identical settings.

5. Conclusion

In this paper, we focus on learning the numerical solution operator of multiscale linear transport equations that involve diffusive scaling. We observed that the vanilla physics-informed DeepONets, when equipped with a modified MLP, may encounter instability issues in maintaining the desired macroscopic behavior. To tackle this challenge, we first introduce the concept of an Asymptotic-Preserving (AP) loss function within the realm of operator learning. Subsequently, drawing inspiration from the heat kernel in the

diffusion equation, we propose an innovative architecture called Convolutional DeepONets. These networks leverage multiple local convolution operations, rather than relying on a global heat kernel, while incorporating pooling and activation operations within each filter layer. As a result, we present two novel variants of Asymptotic-Preserving Convolutional DeepONets (APCONs) based on micro-macro and even–odd decomposition. Significantly, our APCON approaches maintain a consistent parameter count regardless of the grid size, effectively capturing the inherent diffusive behavior in the linear transport problem.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

Acknowledgments

SJ was partially supported by the National Key R&D Program of China (no. 2020YFA0712000), the NSFC grant No. 12031013,and the Shanghai Municipal Science and Technology Major Project (2021SHZDZX0102). Shi Jin is also supported by NSFC grant No. 11871297. Zheng Ma is also supported by NSFC Grant No. 12031013, No. 92270120 and partially supported by Institute of Modern Analysis—A Shanghai Frontier Research Center.

References

[1] François Bouchut, François Golse, Mario Pulvirenti, Kinetic Equations and Asymptotic Theory, Elsevier, 2000.
[2] Giacomo Dimarco, Lorenzo Pareschi, Numerical methods for kinetic equations, Acta Numer. 23 (2014) 369–520.
[3] Shi Jin, Asymptotic preserving (AP) schemes for multiscale kinetic and hyperbolic equations: a review, in: Lecture Notes for Summer School on Methods and Models of Kinetic Theory (M&MKT), Porto Ercole (Grosseto, Italy), Citeseer, 2010, pp. 177–216.
[4] Weinan E, Principles of Multiscale Modeling, Cambridge University Press, 2011.
[5] Christian Beck, Martin Hutzenthaler, Arnulf Jentzen, Benno Kuckuck, An overview on deep learning-based approximation methods for partial differential equations., Discrete Contin. Dyn. Syst. Ser. B 28 (6) (2023).
[6] Zhiqiang Cai, Jingshuang Chen, Min Liu, Least-squares ReLU neural network (LSNN) method for linear advection-reaction equation, J. Comput. Phys. (2021) 110514.
[7] Weinan E, Bing Yu, The deep ritz method: A deep learning-based numerical algorithm for solving variational problems, Commun. Math. Stat. 6 (1) (2018) 1–12.
[8] Yulei Liao, Pingbing Ming, Deep nitsche method: Deep ritz method with essential boundary conditions, Commun. Comput. Phys. 29 (5) (2021) 1365–1384.
[9] Liyao Lyu, Zhen Zhang, Minxin Chen, Jingrun Chen, MIM: A deep mixed residual method for solving high-order partial differential equations, J. Comput. Phys. 452 (2022) 110930.
[10] Maziar Raissi, Paris Perdikaris, George Em Karniadakis, Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations, J. Comput. Phys. 378 (2019) 686–707.
[11] Justin Sirignano, Konstantinos Spiliopoulos, DGM: A deep learning algorithm for solving partial differential equations, J. Comput. Phys. 375 (2018) 1339–1364.
[12] Yaohua Zang, Gang Bao, Xiaojing Ye, Haomin Zhou, Weak adversarial networks for high-dimensional partial differential equations, J. Comput. Phys. (2020) 109409.
[13] Lu Lu, Xuhui Meng, Zhiping Mao, George Em Karniadakis, DeepXDE: A deep learning library for solving differential equations, SIAM Rev. 63 (1) (2021) 208–228.
[14] Zongyi Li, Nikola Borislavov Kovachki, Kamyar Azizzadenesheli, Burigede liu, Kaushik Bhattacharya, Andrew Stuart, Anima Anandkumar, Fourier neural operator for parametric partial differential equations, in: International Conference on Learning Representations, 2021.
[15] Lu Lu, Pengzhan Jin, Guofei Pang, Zhongqiang Zhang, George Em Karniadakis, Learning nonlinear operators via DeepONet based on the universal approximation theorem of operators, Nat. Mach. Intell. 3 (3) (2021) 218–229.
[16] Lulu Zhang, Tao Luo, Yaoyu Zhang, Weinan E, Zhi-Qin John Xu, Zheng Ma, MOD-Net: A machine learning approach via model-operator-data network for solving PDEs, Commun. Comput. Phys. 32 (2) (2022) 299–335.
[17] Zongyi Li, Hongkai Zheng, Nikola Kovachki, David Jin, Haoxuan Chen, Burigede Liu, Kamyar Azizzadenesheli, Anima Anandkumar, Physics-informed neural operator for learning partial differential equations, 2021, arXiv preprint arXiv:2111.03794.
[18] Sifan Wang, Hanwen Wang, Paris Perdikaris, Learning the solution operator of parametric partial differential equations with physics-informed deeponets, Sci. Adv. 7 (40) (2021) eabi8605.
[19] Wei Xiong, Xiaomeng Huang, Ziyang Zhang, Ruixuan Deng, Pei Sun, Yang Tian, Koopman neural operator as a mesh-free solver of non-linear partial differential equations, 2023, arXiv preprint arXiv:2301.10022.
[20] Jiahao Zhang, Shiqi Zhang, Guang Lin, MultiAuto-DeepONet: A multi-resolution autoencoder DeepONet for nonlinear dimension reduction, uncertainty quantification and operator learning of forward and inverse stochastic problems, 2022, arXiv preprint arXiv:2204.03193.
[21] Xinliang Liu, Bo Xu, Lei Zhang, HT-net: Hierarchical transformer based operator learning model for multiscale PDEs, 2022, arXiv preprint arXiv:2210.10890.
[22] Qianying Cao, Somdatta Goswami, George Em Karniadakis, LNO: Laplace neural operator for solving differential equations, 2023, arXiv preprint arXiv:2303.10528.
[23] Wuzhe Xu, Yulong Lu, Li Wang, Transfer learning enhanced deeponet for long-time prediction of evolution equations, in: Proceedings of the AAAI Conference on Artificial Intelligence, Vol. 37, 2023, pp. 10629–10636.
[24] Qin Lou, Xuhui Meng, George Em Karniadakis, Physics-informed neural networks for solving forward and inverse flow problems via the Boltzmann-BGK formulation, J. Comput. Phys. 447 (2021) 110676.
[25] Zheng Chen, Liu Liu, Lin Mu, Solving the linear transport equation by a deep neural network approach, Discrete Contin. Dyn. Syst. Ser. S 15 (4) (2022) 669–686.

[26] Hyung Ju Hwang, Jin Woo Jang, Hyeontae Jo, Jae Yong Lee, Trend to equilibrium for the kinetic Fokker-Planck equation via the neural network approach, J. Comput. Phys. 419 (2020) 109665.
[27] Shi Jin, Zheng Ma, Keke Wu, Asymptotic-preserving neural networks for multiscale time-dependent linear transport equations, J. Sci. Comput. 94 (3) (2023) 57.
[28] Yulong Lu, Li Wang, Wuzhe Xu, Solving multiscale steady radiative transfer equation using neural networks with uniform stability, Res. Math. Sci. 9 (3) (2022) 1–29.
[29] Hongyan Li, Song Jiang, Wenjun Sun, Liwei Xu, Guanyu Zhou, A model-data asymptotic-preserving neural network method based on micro-macro decomposition for gray radiative transfer equations, 2022, arXiv preprint arXiv:2212.05523.
[30] Giulia Bertaglia, Asymptotic-preserving neural networks for hyperbolic systems with diffusive scaling, in: Advances in Numerical Methods for Hyperbolic Balance Laws and Related Problems, Springer Nature Switzerland, 2023, pp. 23–48.
[31] Giulia Bertaglia, Chuan Lu, Lorenzo Pareschi, Xueyu Zhu, Asymptotic-preserving neural networks for multiscale hyperbolic models of epidemic spread, Math. Models Methods Appl. Sci. 32 (10) (2022) 1949–1985.
[32] Zhengyi Li, Yanli Wang, Hongsheng Liu, Zidong Wang, Bin Dong, Solving Boltzmann equation with neural sparse representation, 2023, arXiv preprint arXiv:2302.09233.
[33] Shi Jin, Zheng Ma, Keke Wu, Asymptotic-preserving neural networks for multiscale kinetic equations, 2023, arXiv preprint arXiv:2306.15381.
[34] Mohammed Lemou, Luc Mieussens, A new asymptotic preserving scheme based on micro-macro formulation for linear kinetic equations in the diffusion limit, SIAM J. Sci. Comput. 31 (1) (2008) 334–368.
[35] Elmer Eugene Lewis, Warren F. Miller, Computational Methods of Neutron Transport, John Wiley and Sons, Inc., New York, NY, 1984.
[36] Walter A. Strauss, Partial Differential Equations: An Introduction, John Wiley & Sons, 2007.
[37] Evans C. Lawrence, Partial Differential Equations (Second Edition), American Mathematical Society, 2010.
[38] Sifan Wang, Yujun Teng, Paris Perdikaris, Understanding and mitigating gradient flow pathologies in physics-informed neural networks, SIAM J. Sci. Comput. 43 (5) (2021) A3055–A3081.
[39] Jimmy Lei Ba, Jamie Ryan Kiros, Geoffrey E. Hinton, Layer normalization, 2016, arXiv preprint arXiv:1607.06450.
[40] Tianping Chen, Hong Chen, Universal approximation to nonlinear operators by neural networks with arbitrary activation functions and its application to dynamical systems, IEEE Trans. Neural Netw. 6 (4) (1995) 911–917.
[41] Diederik P. Kingma, Jimmy Ba, Adam: A method for stochastic optimization, in: ICLR, 2015.